

循序渐进，学习开发 xv6-riscv

第 6 章 协作式多任务

汪辰



目录

01

协作式多任务简介

02

协作式多任务设计

03

深入剖析协作式多任务



01

协作式多任务简介



多任务的实现方式

- **协作式多任务 (Cooperative Multi-Tasking)**: 正在处理器上运行的用户任务主动放弃处理器, 将处理器的控制权交还给操作系统, 这样操作系统就可以执行相关的代码并选择和运行其他的用户任务。也就是说, 多个任务相互协作, 交替运行和放弃处理器, 达到整体系统上多个任务并发运行的效果。
- **抢占式多任务 (Preemptive Multi-Tasking)**: 和协作方式相反, 操作系统并不信任用户任务会合理地主动放弃处理器, 所以内核通过其他手段强制剥夺正在处理器上运行的用户任务对处理器的使用权, 将处理器分配给其他的任务实现交替运行和放弃处理器, 同样达到整体系统上多个任务并发运行的效果。

多任务的实现方式

	协作式多任务	抢占式多任务
优点	• 实现简单 • 任务切换发生在已知、安全的点。	• 健壮性高：一个任务的崩溃、死循环或恶意行为通常不会影响其他任务的执行，因为操作系统可以强制将其切换出去。 • 公平性与响应性：操作系统可以确保所有任务都能分到 CPU 时间，并能快速响应高优先级事件（如用户交互）。
缺点	• 一个不合作的任务可以挂起整个系统。如果一个任务陷入死循环且不主动让出 CPU，操作系统将无法恢复控制，其他所有任务都会被“饿死”。 • 响应性差，高优先级的任务无法及时获得 CPU。	• 实现复杂，需要硬件支持（如定时器）。 • 需要仔细处理共享数据的同步问题（竞态条件）。
典型应用	早期的 Mac OS、Windows 3.x，以及一些嵌入式系统	所有现代通用操作系统，以及现代嵌入式 RTOS。

A decorative graphic on the left side of the slide. It features two concentric circles with a light blue glow. A thin grey arc with dots at its ends is positioned around the circles. In the top right corner, there is a solid blue rectangle.

02

协作式多任务设计

协作式多任务设计

```
// Give up the CPU for one scheduling round.  
void  
yield(void)  
{  
    struct proc *p = myproc();  
    p->state = RUNNABLE;  
    sched();  
}
```

协作式多任务设计

```
// Give up the CPU for one scheduling round.
```

```
void
```

```
yield(void)
```

```
{
```

```
    struct proc *p = myproc();
```

```
    p->state = RUNNABLE;
```

```
    sched();
```

```
}
```

```
// Return the current struct proc *, or zero if none.
```

```
struct proc*
```

```
myproc(void)
```

```
{
```

```
    struct cpu *c = mycpu();
```

```
    struct proc *p = c->proc;
```

```
    return p;
```

```
}
```


协作式多任务设计

```
// Give up the CPU for one scheduling round.  
void  
yield(void)  
{  
    struct proc *p = myproc();  
    p->state = RUNNABLE;  
    sched();  
}
```

协作式多任务设计

```
// Give up the CPU for one scheduling round.
```

```
void
```

```
yield(void)
```

```
{
```

```
    struct proc *p = myproc();
```

```
    p->state = RUNNABLE;
```

```
    sched();
```

```
}
```

```
// Switch to scheduler.
```

```
void
```

```
sched(void)
```

```
{
```

```
    struct proc *p = myproc();
```

```
    if(p->state == RUNNING)
```

```
        panic("sched RUNNING");
```

```
    swtch(&p->context, &mycpu()->context);
```

```
}
```

协作式多任务设计

```
// Give up the CPU for one scheduling round.
```

```
void
```

```
yield(void)
```

```
{
```

```
    struct proc *p = myproc();
```

```
    p->state = RUNNABLE;
```

```
    sched();
```

```
}
```

```
// Switch to scheduler.
```

```
void
```

```
sched(void)
```

```
{
```

```
    struct proc *p = myproc();
```

```
    if(p->state == RUNNING)
```

```
        panic("sched RUNNING");
```

```
    swtch(&p->context, &mycpu()->context);
```

```
}
```

协作式多任务设计

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

协作式多任务设计

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

协作式多任务设计

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

xv6 kernel is booting

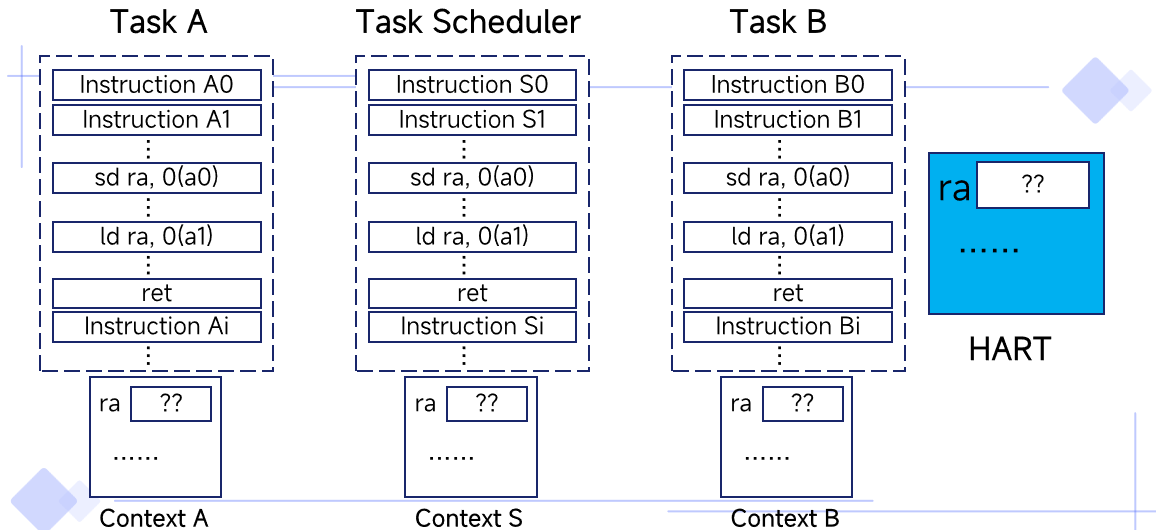
```
Task 0: Created!
Task 0: Running...
Task 1: Created!
Task 1: Running...
Task 0: Running...
Task 1: Running...
Task 0: Running...
Task 1: Running...
Task 0: Running...
Task 1: Running...
```



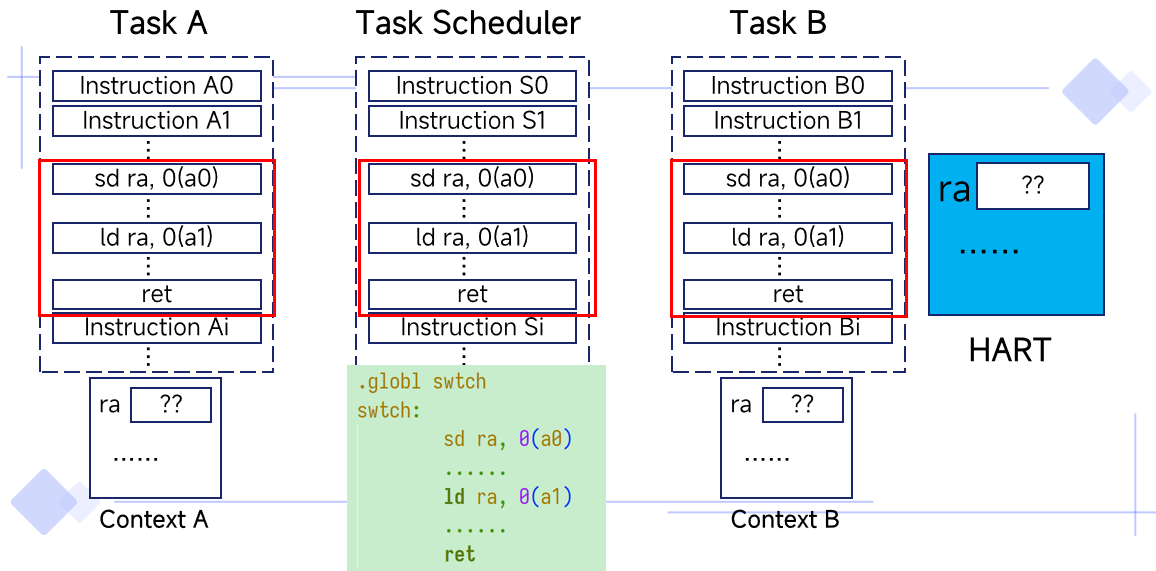
03

深入剖析协作式多任务

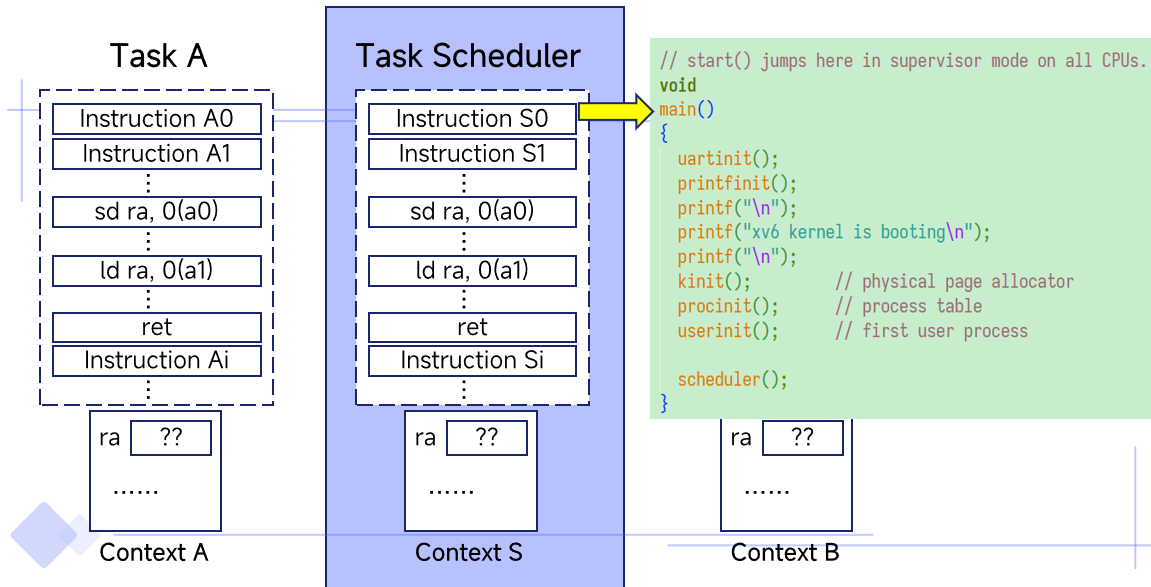
深入剖析协作式多任务



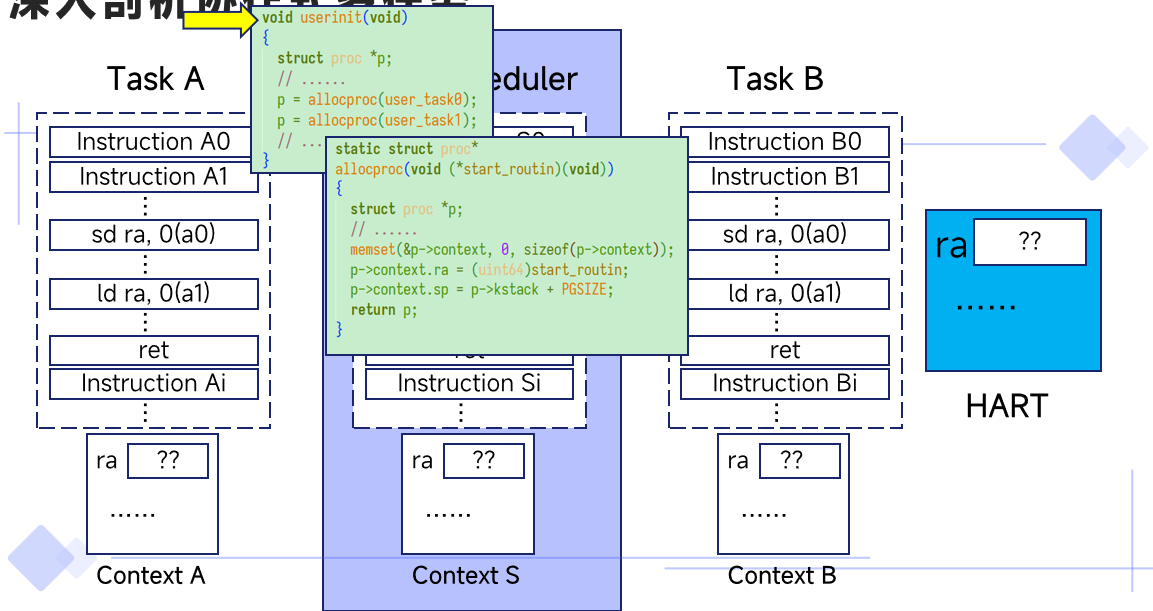
深入剖析协作式多任务



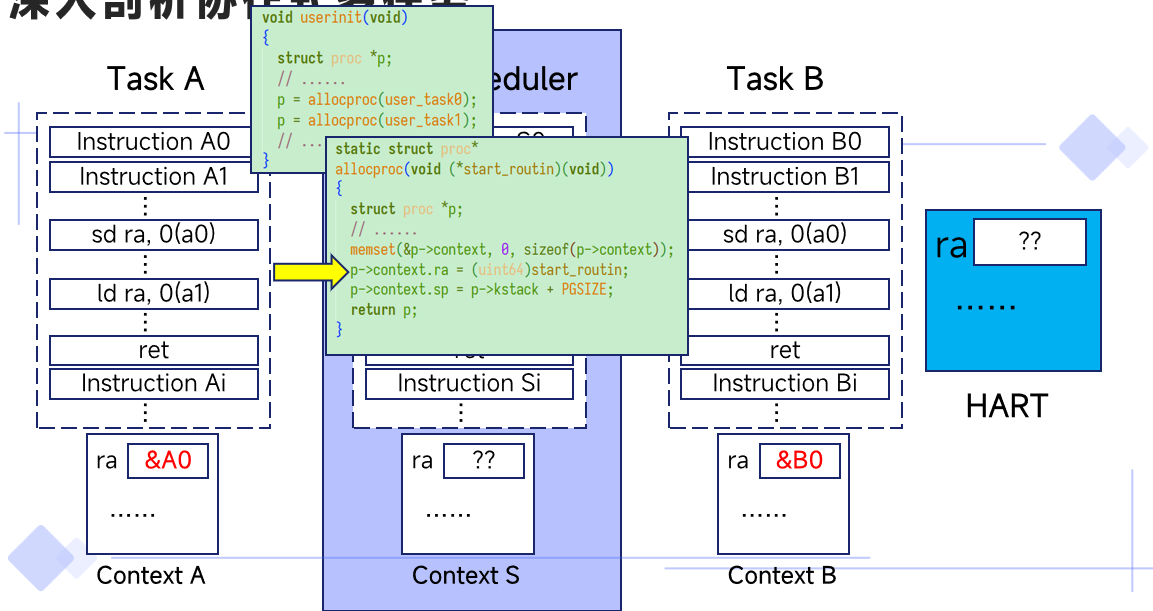
深入剖析协作式多任务



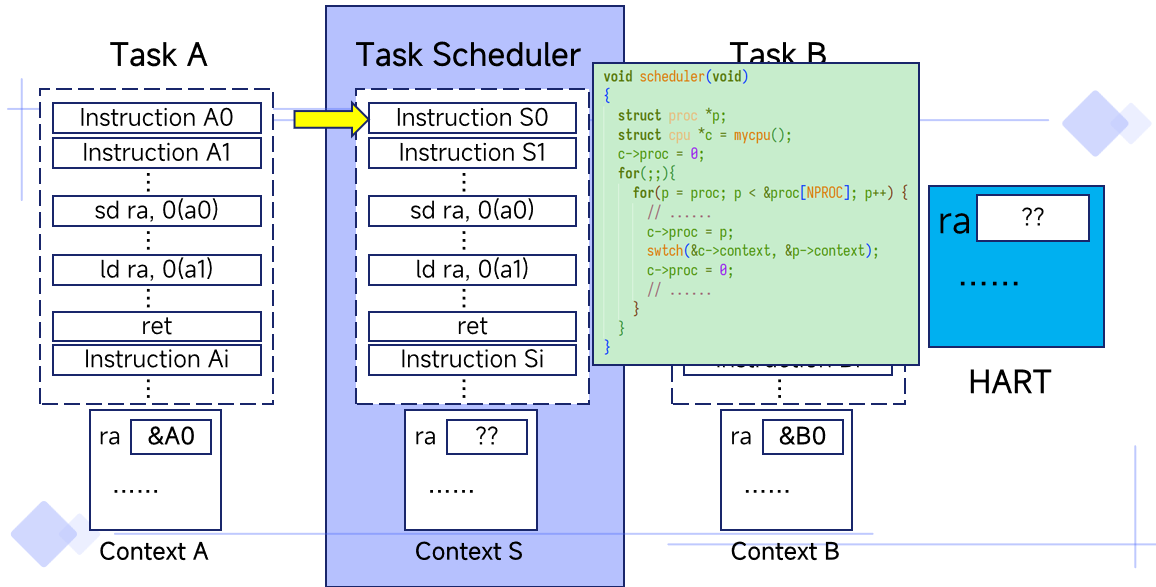
深入剖析协作式多任务



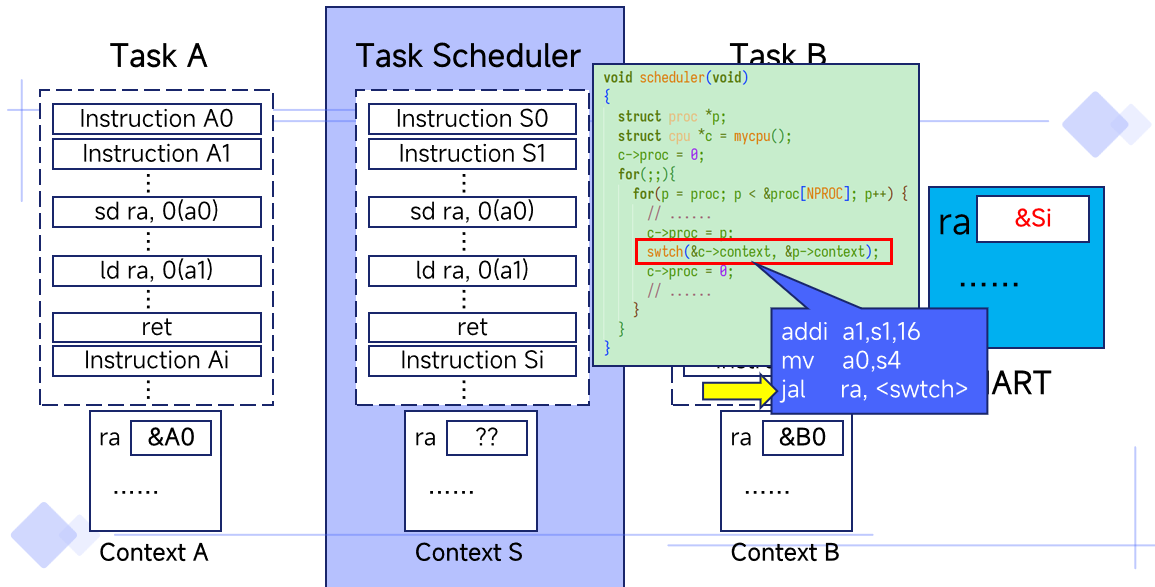
深入剖析协作式多任务



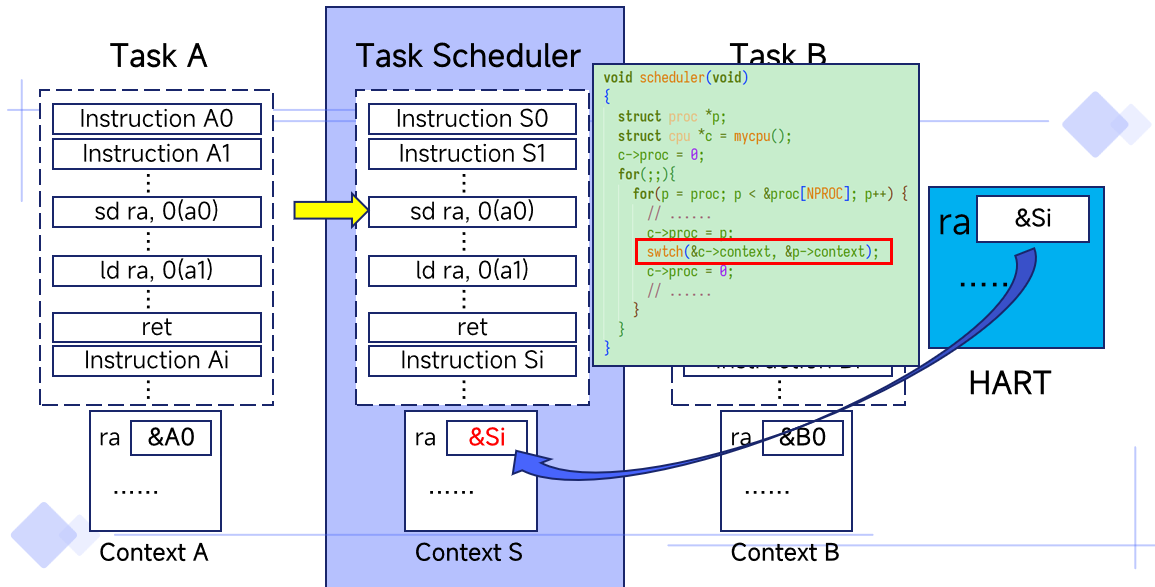
深入剖析协作式多任务



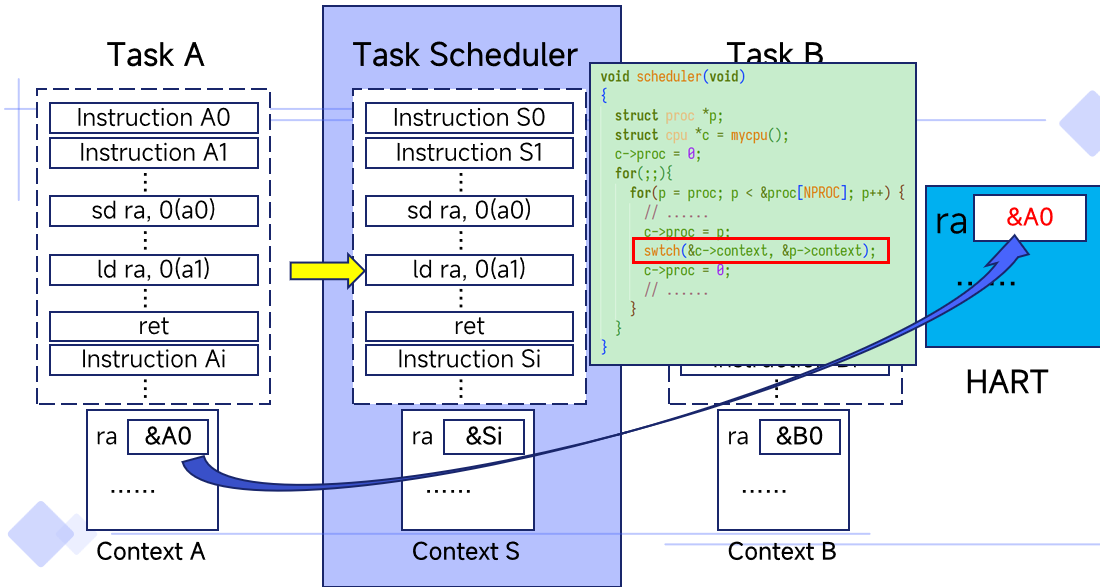
深入剖析协作式多任务



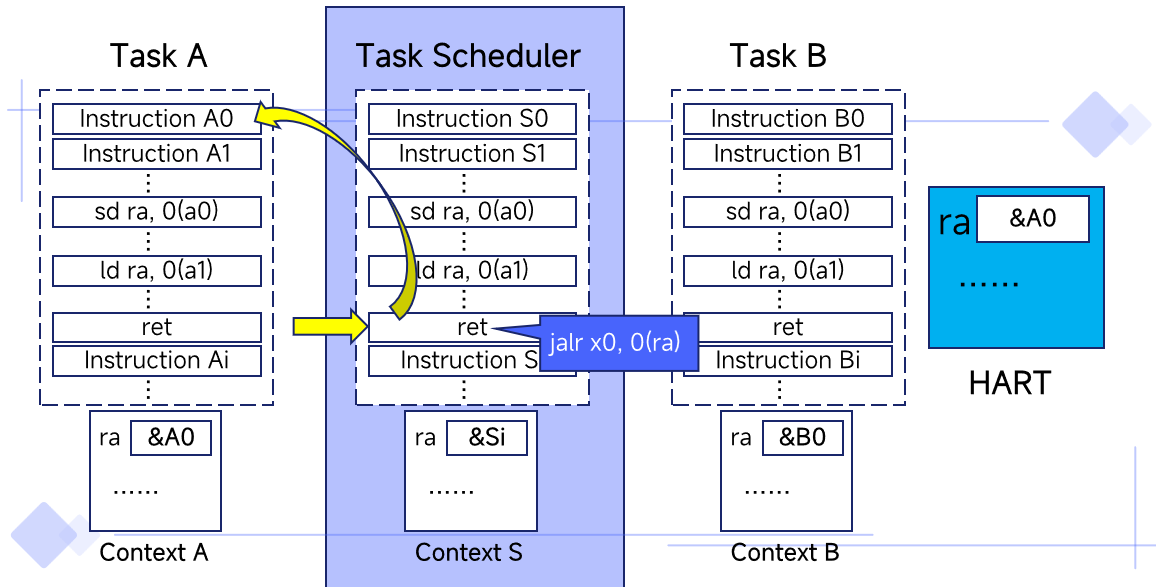
深入剖析协作式多任务



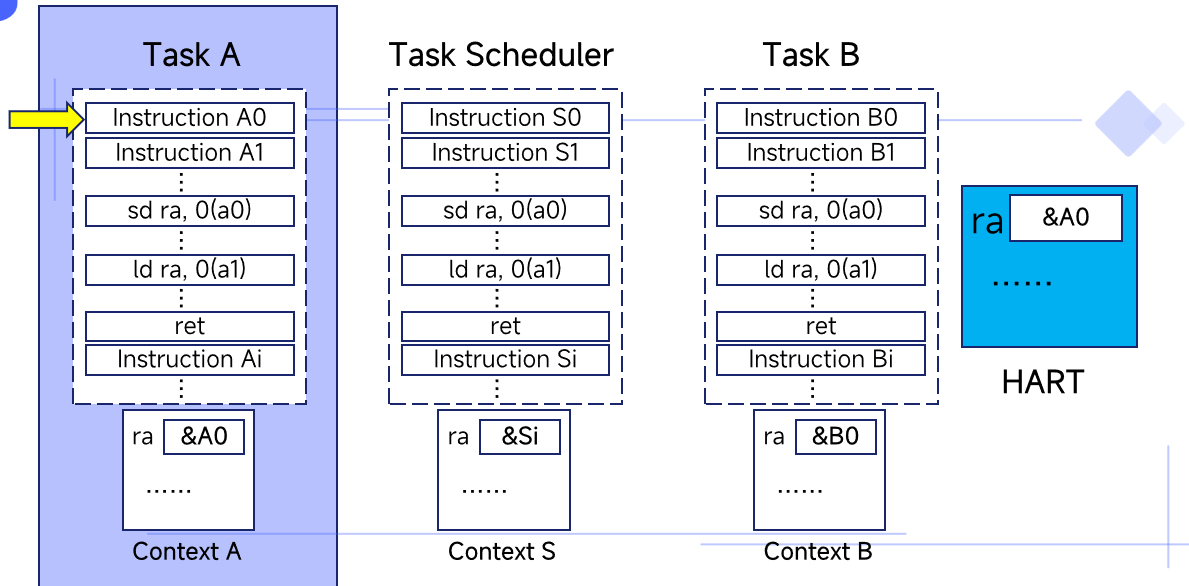
深入剖析协作式多任务



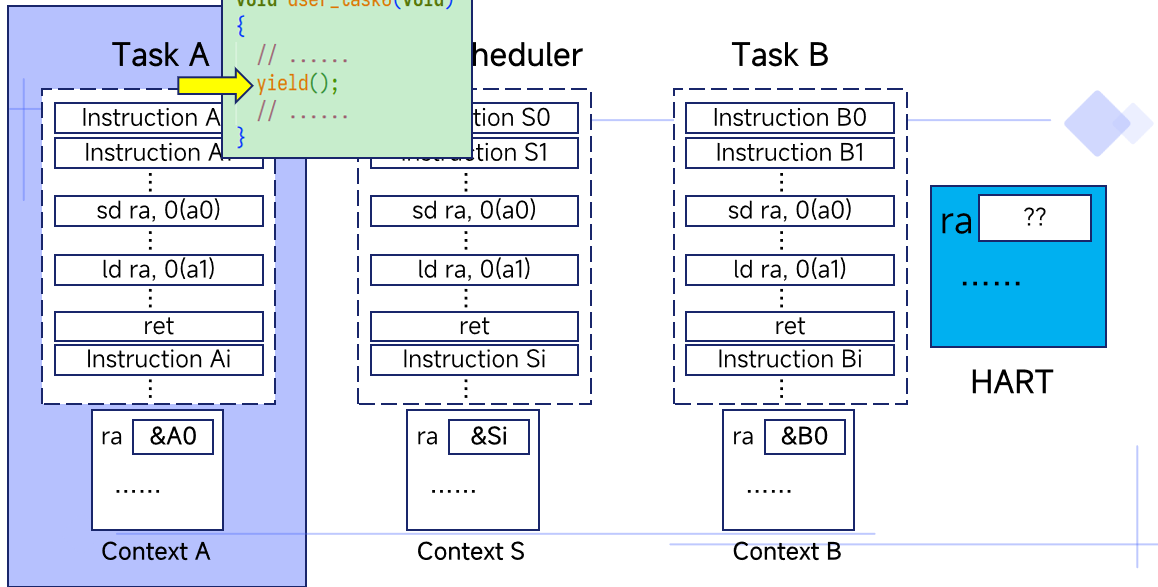
深入剖析协作式多任务



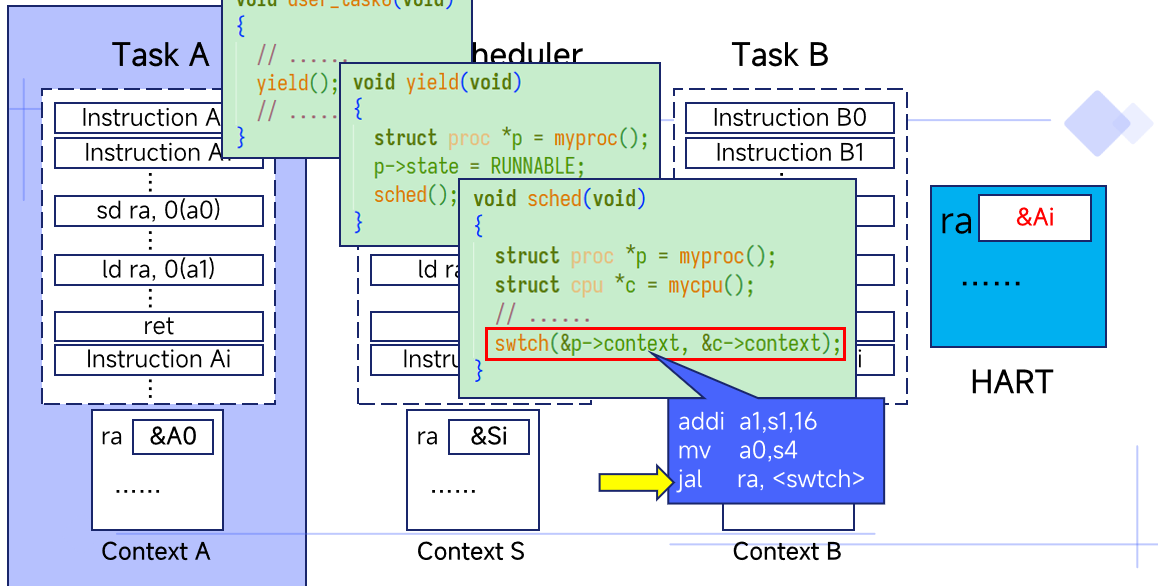
深入剖析协作式多任务



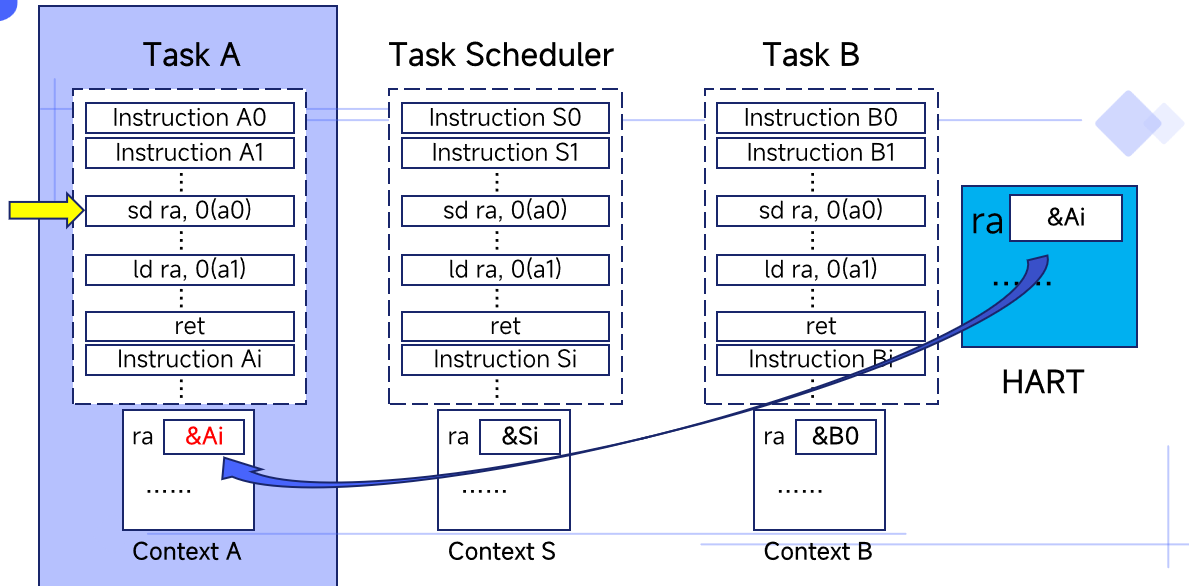
深入剖析协作式多任务



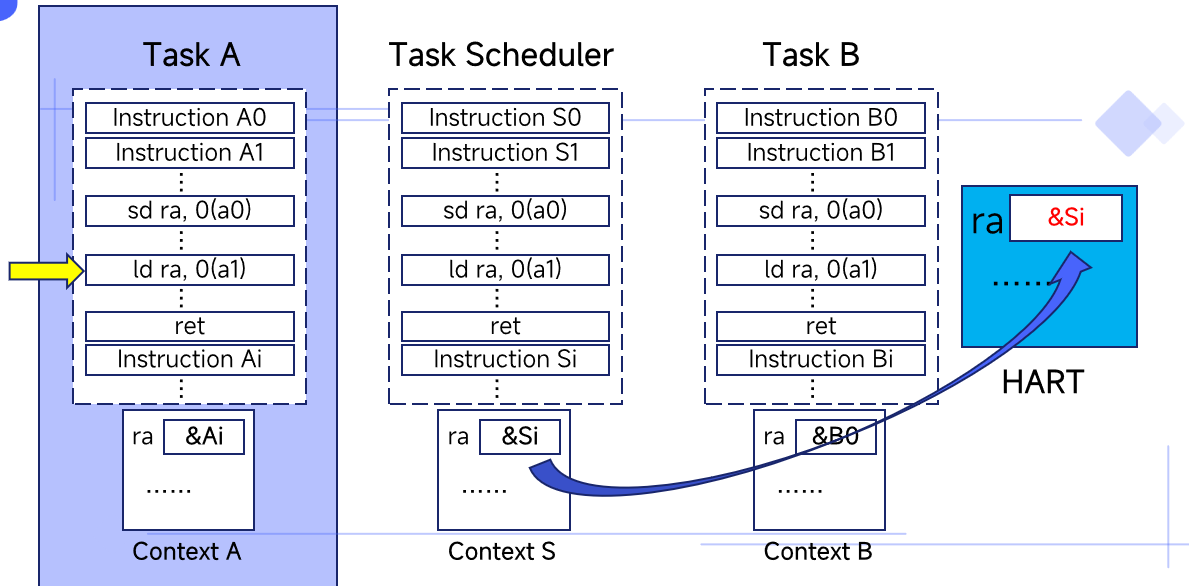
深入剖析协作式多任务



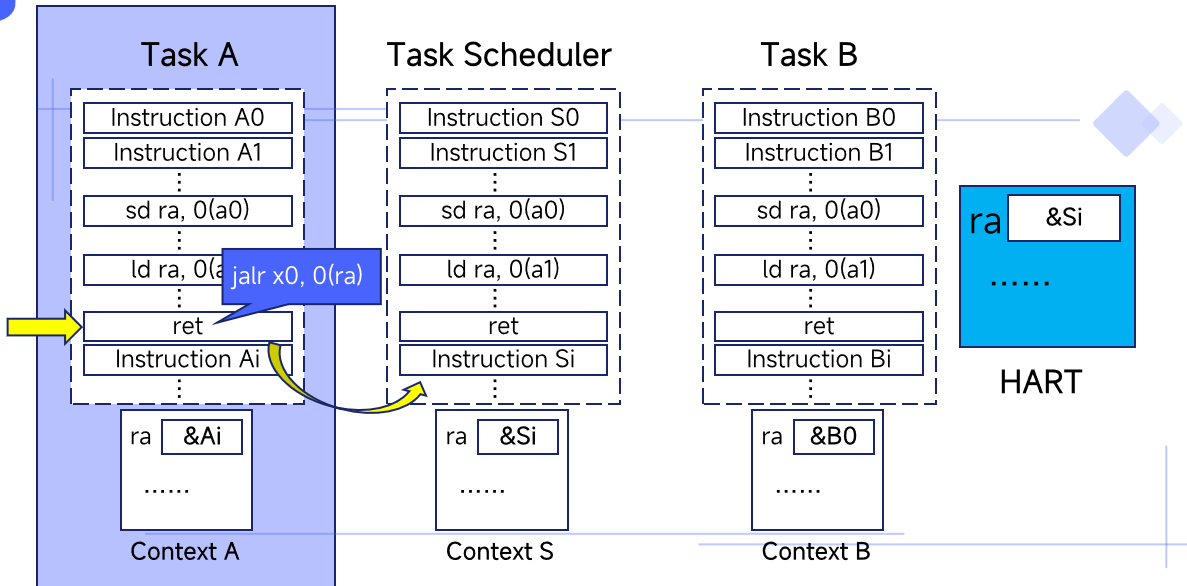
深入剖析协作式多任务



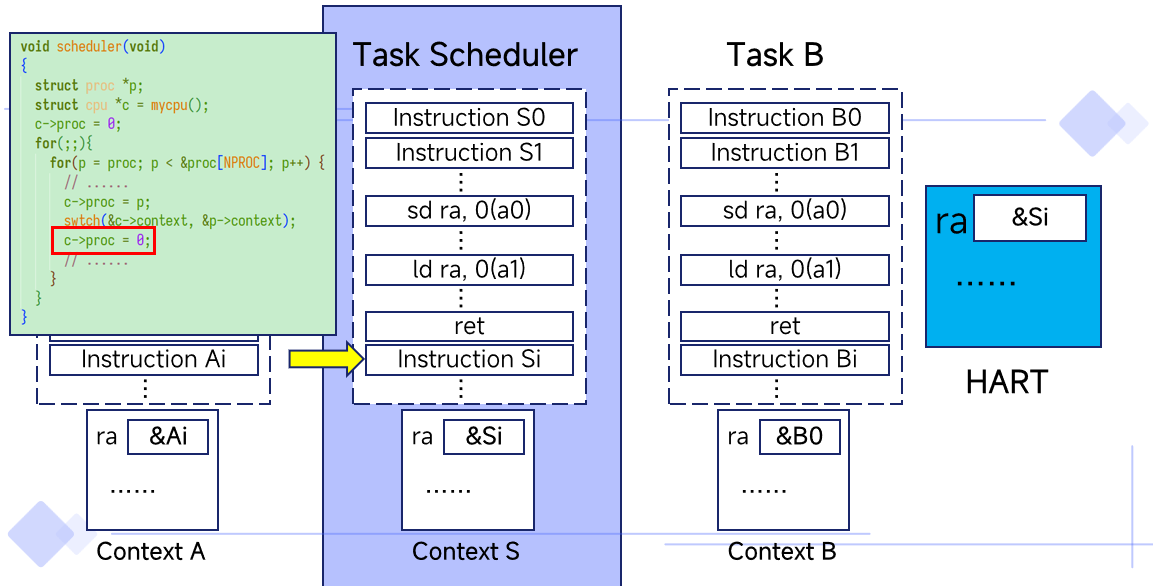
深入剖析协作式多任务



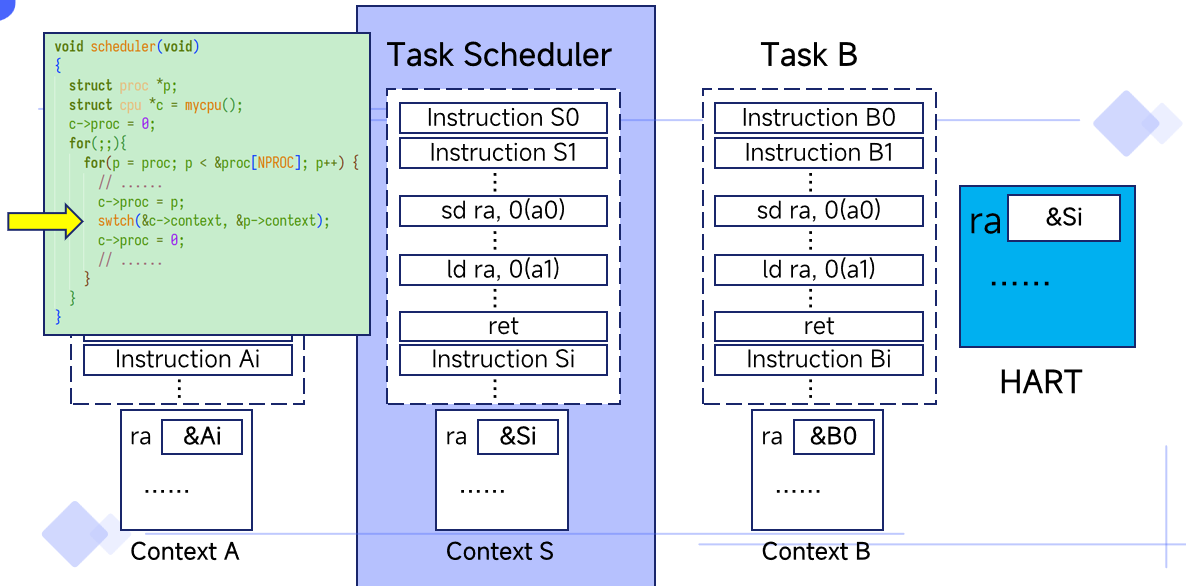
深入剖析协作式多任务



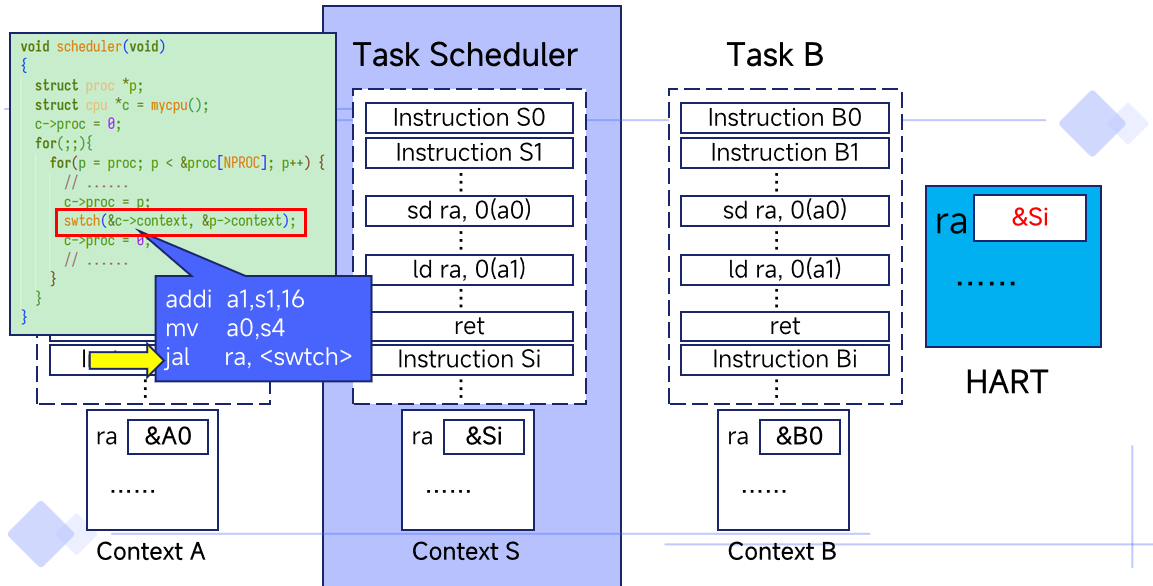
深入剖析协作式多任务



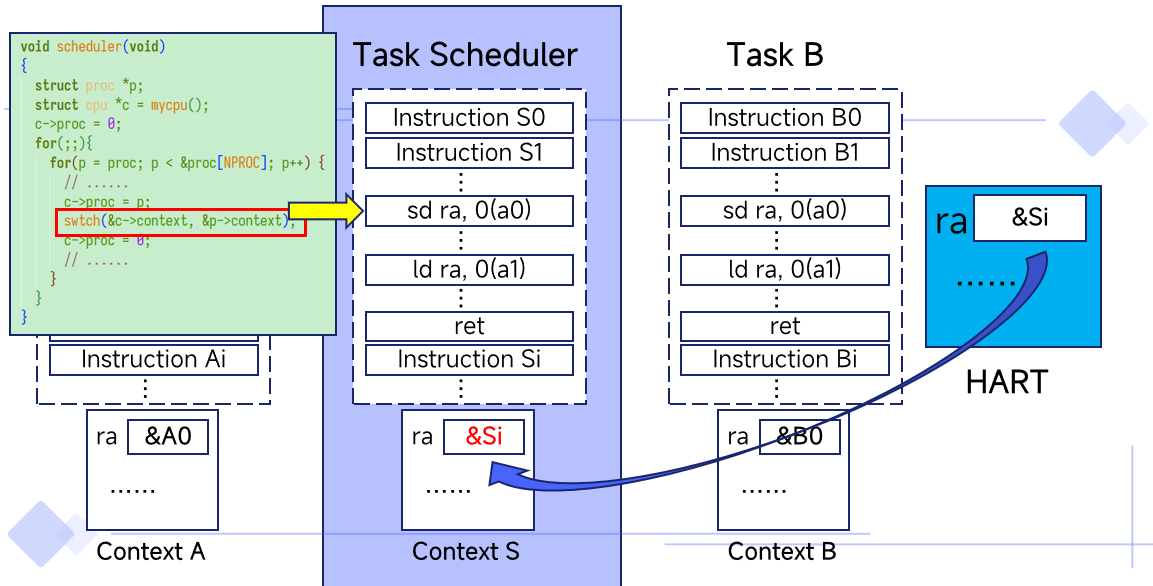
深入剖析协作式多任务



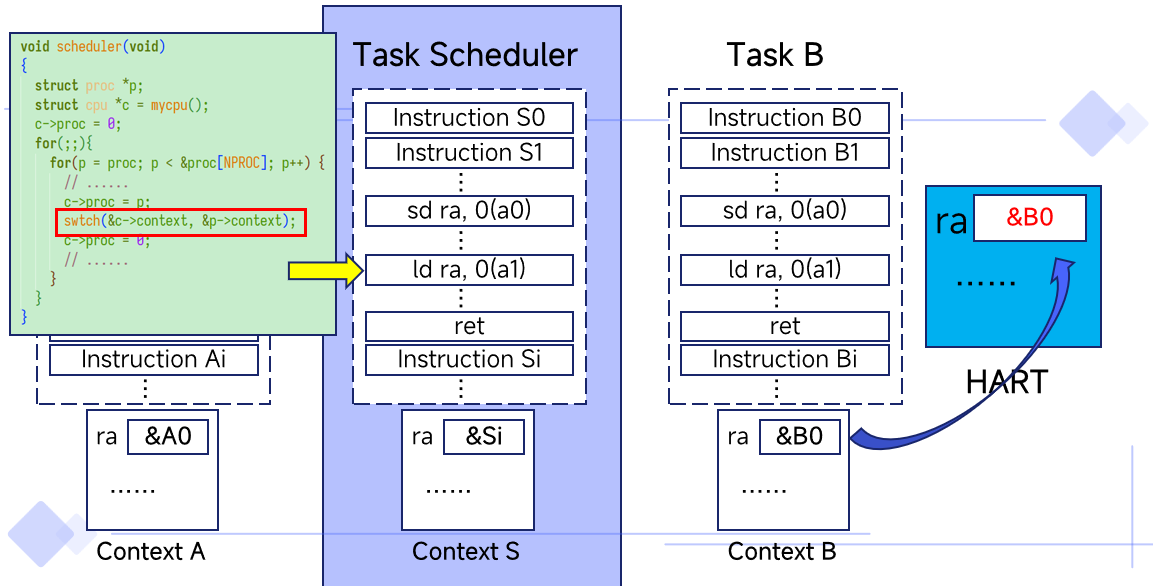
深入剖析协作式多任务



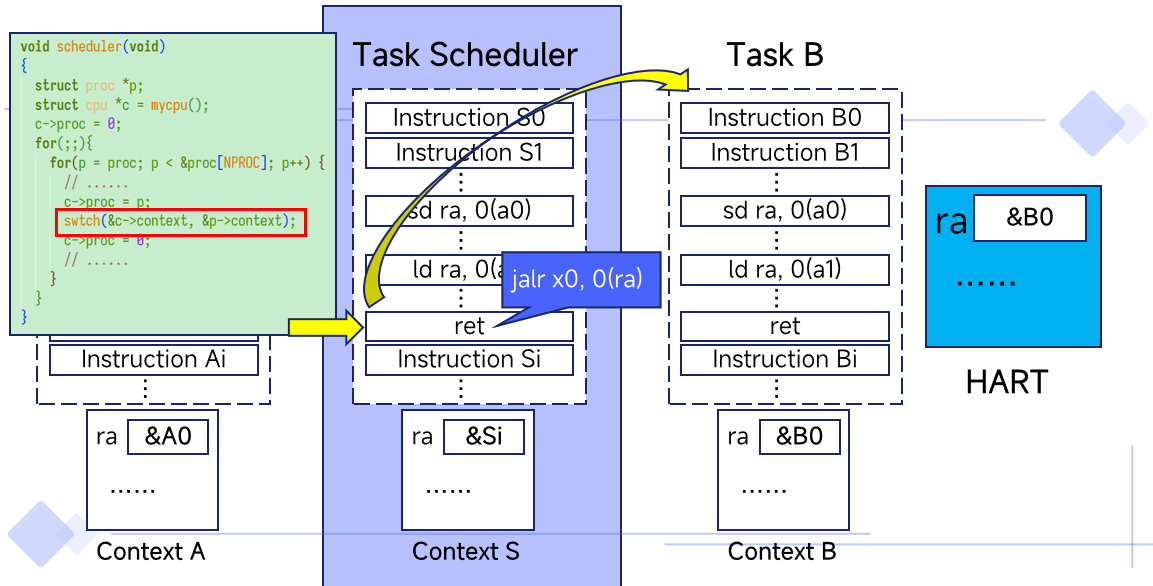
深入剖析协作式多任务



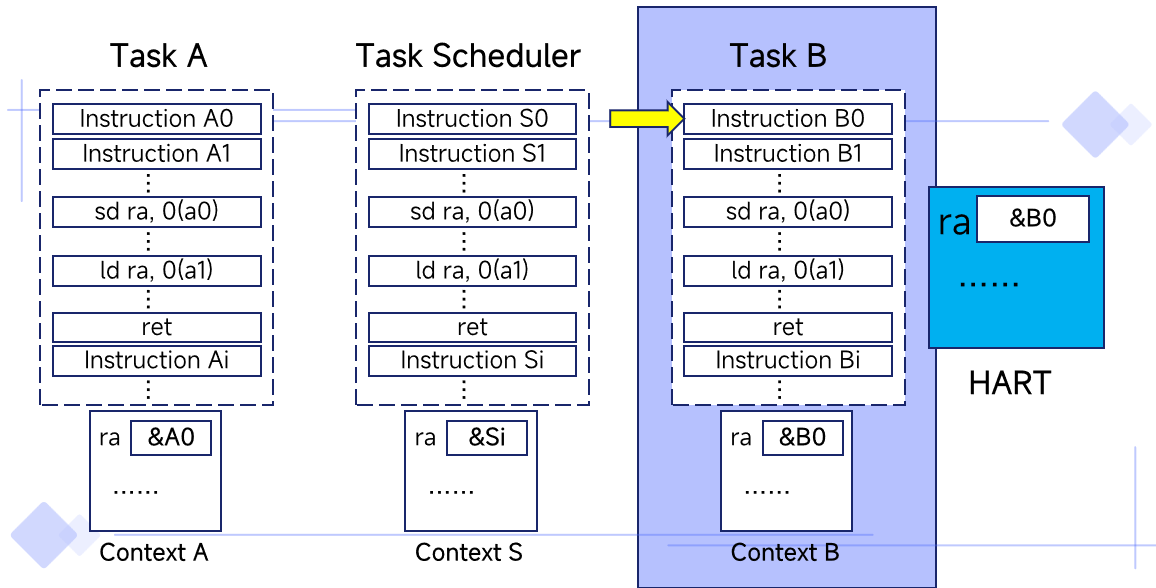
深入剖析协作式多任务



深入剖析协作式多任务



深入剖析协作式多任务



深入剖析协作式多任务 - 演示

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```



```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

xv6 kernel is booting

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

深入剖析协作式多任务 - 演示

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

xv6 kernel is booting

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```


深入剖析协作式多任务 - 演示

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

xv6 kernel is booting
Task 0: Created!

深入剖析协作式多任务 - 演示

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

xv6 kernel is booting

Task 0: Created!
Task 0: Running...

深入剖析协作式多任务 - 演示

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```



```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```



```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

xv6 kernel is booting

Task 0: Created!
Task 0: Running...

深入剖析协作式多任务 - 演示

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

xv6 kernel is booting

Task 0: Created!
Task 0: Running...

深入剖析协作式多任务 - 演示

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

xv6 kernel is booting

Task 0: Created!
Task 0: Running...

深入剖析协作式多任务 - 演示

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

xv6 kernel is booting

Task 0: Created!
Task 0: Running...

深入剖析协作式多任务 - 演示

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

xv6 kernel is booting

Task 0: Created!
Task 0: Running...
Task 1: Created!

深入剖析协作式多任务 - 演示

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```



```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```



```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

xv6 kernel is booting

```
Task 0: Created!
Task 0: Running...
Task 1: Created!
Task 1: Running...
```


深入剖析协作式多任务 - 演示

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

xv6 kernel is booting

```
Task 0: Created!
Task 0: Running...
Task 1: Created!
Task 1: Running...
```

深入剖析协作式多任务 - 演示

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

xv6 kernel is booting

Task 0: Created!
Task 0: Running...
Task 1: Created!
Task 1: Running...

深入剖析协作式多任务 - 演示

```
void user_task0(void)
```

```
{  
    printf("Task 0: Created!\n");  
    while (1) {  
        printf("Task 0: Running...\n");  
        task_delay(DELAY);  
        yield();  
    }  
}
```

```
void scheduler(void)
```

```
{  
    struct proc *p;  
    struct cpu *c = mycpu();  
    c->proc = 0;  
    for(;;){  
        for(p = proc; p < &proc[NPROC]; p++) {  
            // .....  
            c->proc = p;  
            switch(&c->context, &p->context);  
            c->proc = 0;  
            // .....  
        }  
    }  
}
```

```
void user_task1(void)
```

```
{  
    printf("Task 1: Created!\n");  
    while (1) {  
        printf("Task 1: Running...\n");  
        task_delay(DELAY);  
        yield();  
    }  
}
```

xv6 kernel is booting

```
Task 0: Created!  
Task 0: Running...  
Task 1: Created!  
Task 1: Running...
```

深入剖析协作式多任务 - 演示

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

xv6 kernel is booting

```
Task 0: Created!
Task 0: Running...
Task 1: Created!
Task 1: Running...
```

深入剖析协作式多任务 - 演示

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```



```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```



```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```



xv6 kernel is booting

```
Task 0: Created!
Task 0: Running...
Task 1: Created!
Task 1: Running...
Task 0: Running...
```

深入剖析协作式多任务 - 演示

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```



```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```



```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```



xv6 kernel is booting

```
Task 0: Created!
Task 0: Running...
Task 1: Created!
Task 1: Running...
Task 0: Running...
```

深入剖析协作式多任务 - 演示

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

xv6 kernel is booting

```
Task 0: Created!
Task 0: Running...
Task 1: Created!
Task 1: Running...
Task 0: Running...
```

深入剖析协作式多任务 - 演示

```
void user_task0(void)
```

```
{  
    printf("Task 0: Created!\n");  
    while (1) {  
        printf("Task 0: Running...\n");  
        task_delay(DELAY);  
        yield();  
    }  
}
```

```
void scheduler(void)
```

```
{  
    struct proc *p;  
    struct cpu *c = mycpu();  
    c->proc = 0;  
    for(;;){  
        for(p = proc; p < &proc[NPROC]; p++) {  
            // .....  
            c->proc = p;  
            switch(&c->context, &p->context);  
            c->proc = 0;  
            // .....  
        }  
    }  
}
```

```
void user_task1(void)
```

```
{  
    printf("Task 1: Created!\n");  
    while (1) {  
        printf("Task 1: Running...\n");  
        task_delay(DELAY);  
        yield();  
    }  
}
```

xv6 kernel is booting

```
Task 0: Created!  
Task 0: Running...  
Task 1: Created!  
Task 1: Running...  
Task 0: Running...
```


深入剖析协作式多任务 - 演示

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

xv6 kernel is booting

```
Task 0: Created!
Task 0: Running...
Task 1: Created!
Task 1: Running...
Task 0: Running...
```

深入剖析协作式多任务 - 演示

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        for(p = proc; p < &proc[NPROC]; p++) {
            // .....
            c->proc = p;
            switch(&c->context, &p->context);
            c->proc = 0;
            // .....
        }
    }
}
```

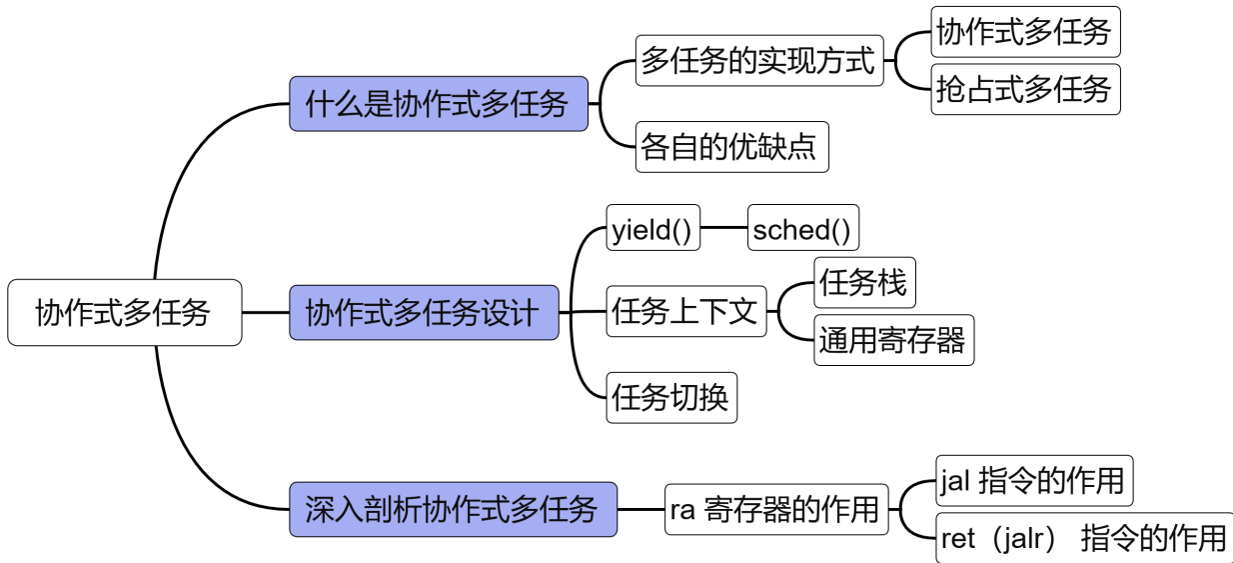
```
void user_task1(void)
{
    printf("Task 1: Created!\n");
    while (1) {
        printf("Task 1: Running...\n");
        task_delay(DELAY);
        yield();
    }
}
```

xv6 kernel is booting

```
Task 0: Created!
Task 0: Running...
Task 1: Created!
Task 1: Running...
Task 0: Running...
Task 1: Running...
```



本章总结



谢谢

